

Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF
ARTS, SCIENCE AND COMMERCE
(EMPOWERED AUTONOMOUS)

Data Engineering



Name: Surajkumar Yadav

Roll no: 10302

Class: MSc. Data Science and Artificial Intelligence Part I



Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify Surajkumar Yadav of MSc. Data Science and Artificial Intelligence Roll No 10302 has successfully completed the practical of Data Engineering during the Academic Year 2024-2025.

Date:

**(Prof. Neha Ansari)
Prof-In-Charge**

External Examiner

INDEX

Sr. No.	Practical	Date	Sign
1	A. Data Extraction Using Web Scraping B. Perform data extraction from xml file to dataframe/csv	20/01/2025	
2	Create ML pipeline(workflow) using python libraries	20/01/2025	
3	Working with NoSQL database - HBase	13/02/2025	
4	Simulating Datawarehouse environment	17/02/2025	
5	A. Working with different types of tables in hive B. Demonstrating table partitioning, clustering (Bucketing in hive)	20/02/2025	
6	Demonstrating Publisher/Subscriber messaging system using Kafka	03/03/2025	
7	A. Write a simple PySpark Driver Program B. Working with different transformation and actions on RDD by fetching data from an external file	04/03/2025	
8	Data Exploration using Spark SQL	06/03/2025	

Practical 1-A: Data Extraction Using Web Scraping

```
[ ] import pandas as pd
    from bs4 import BeautifulSoup
    import requests as rs
```

```
[ ] url = "https://www.worldometers.info/world-population/population-by-country/"
    src = rs.get(url)
    print("Status code:", src.status_code)
```

🔗 Status code: 200

```
[ ] soup = BeautifulSoup(src.content, features = "lxml") #parsing tg
    soup
```

🔗

```
<!DOCTYPE html>
<!--[if IE 8]> <html lang="en" class="ie8"> <![endif]--><!--[if IE 9]> <html lang="en" class="ie9"> <![endif]--><!--[if !IE]><
charset="utf-8"/> <meta content="IE=edge" http-equiv="X-UA-Compatible"/> <meta content="width=device-width, initial-scale=1" r
Worldometer</title><meta content="List of countries and dependencies in the world ranked by population, from the most populat
density, population density, urbanization, urban population, share of world population." name="description"/><!-- Favicon --><
type="image/x-icon"/><link href="/favicon/apple-icon-57x57.png" rel="apple-touch-icon" sizes="57x57"/><link href="/favicon/app
sizes="60x60"/><link href="/favicon/apple-icon-72x72.png" rel="apple-touch-icon" sizes="72x72"/><link href="/favicon/apple-icc
<link href="/favicon/apple-icon-114x114.png" rel="apple-touch-icon" sizes="114x114"/><link href="/favicon/apple-icon-120x120.p
href="/favicon/apple-icon-144x144.png" rel="apple-touch-icon" sizes="144x144"/><link href="/favicon/apple-icon-152x152.png" re
href="/favicon/apple-icon-180x180.png" rel="apple-touch-icon" sizes="180x180"/><link href="/favicon/android-icon-192x192.png"
href="/favicon/favcon-32x32.png" rel="icon" sizes="32x32" type="image/png"/><link href="/favicon/favcon-96x96.png" rel="icon
href="/favicon/favcon-16x16.png" rel="icon" sizes="16x16" type="image/png"/><link href="/favicon/manifest.json" rel="manifest
TileColor"/><meta content="/favicon/ms-icon-144x144.png" name="msapplication-TileImage"/><meta content="#ffffff" name="theme-c
content="https://www.worldometers.info/favicon/apple-touch-icon" rel="apple-touch-icon" sizes="57x57"/><!-- Bootstrap --><link href="/css/boots
```

```
[ ] soup.prettify()
```

🔗

```
'<!DOCTYPE html>\n<!--[if IE 8]> <html lang="en" class="ie8"> <![endif]-->\n<!--[if IE 9]> <html lang="en" class="ie9"> <![endif]
[endif]-->\n<head>\n  <meta charset="utf-8"/>\n  <meta content="IE=edge" http-equiv="X-UA-Compatible"/>\n  <meta content="width:
<title>\n    Population by Country (2025) - Worldometer\n  </title>\n  <meta content="List of countries and dependencies in the w
Growth rate, median age, fertility rate, area, density, population density, urbanization, urban population, share of world popul.
<link href="/favicon/favcon.ico" rel="shortcut icon" type="image/x-icon"/>\n  <link href="/favicon/apple-icon-57x57.png" rel="a
href="/favicon/apple-icon-60x60.png" rel="apple-touch-icon" sizes="60x60"/>\n  <link href="/favicon/apple-icon-72x72.png" rel="a
href="/favicon/apple-icon-76x76.png" rel="apple-touch-icon" sizes="76x76"/>\n  <link href="/favicon/apple-icon-114x114.png" rel=
href="/favicon/apple-icon-120x120.png" rel="apple-touch-icon" sizes="120x120"/>\n  <link href="/favicon/apple-icon-144x144.png" r
```

```
[ ] soup.body()
```

🔗

```
[<script async="" src="https://www.googletagmanager.com/gtag/js?id=G-ZDP3BFSX60"></script>,
<script> window.dataLayer = window.dataLayer || []; function gtag(){dataLayer.push(arguments);} gtag('js', new Date()); gtag('config
<script async="" crossorigin="anonymous" src="https://pagead2.googlesyndication.com/pagead/js/adsbygoogle.js?client=ca-pub-37016976;
<script async="" src="https://securepubads.g.doubleclick.net/tag/js/gpt.js"></script>,
<link crossorigin="" href="https://a.pub.network/" rel="preconnect"/>,
<link crossorigin="" href="https://b.pub.network/" rel="preconnect"/>,
<link crossorigin="" href="https://c.pub.network/" rel="preconnect"/>,
<link crossorigin="" href="https://d.pub.network/" rel="preconnect"/>,
<link crossorigin="" href="https://c.amazon-adsystem.com" rel="preconnect"/>,
<link crossorigin="" href="https://s.amazon-adsystem.com" rel="preconnect"/>,
<link crossorigin="" href="https://btloader.com/" rel="preconnect"/>,
<link crossorigin="" href="https://www.googletagmanager.com/gtag/js?id=G-ZDP3BFSX60"></script>,
<script> window.dataLayer = window.dataLayer || []; function gtag(){dataLayer.push(arguments);} gtag('js', new Date()); gtag('config
```

```
[ ] t1 = soup.find('table')
    t1
```

🔗

```
<table cellpadding="0" class="table table-striped table-bordered" id="example2" width="100%"> <thead> <tr> <th>#</th> <th>Country
</th> <th>Yearly<br> Change</th> <th>Net<br> Change</th> <th>Density<br> (P/Km²)</th> <th>Land Area<br> (Km²)</th> <th>Migrant<br>
<br> Age</th> <th>Urban<br> Pop %</th> <th>World<br> Share</th> </tr> </thead> <tbody> <tr> <td>1</td> <td style="font-weight: l
href="/world-population/india-population/">India</td> <td style="font-weight: bold;">1,450,935,791</td> <td>0.89 %</td> <td>12
<td>-630,830</td> <td>2.0</td> <td>28</td> <td>37 %</td> <td>17.78 %</td> </tr> <tr> <td>2</td> <td style="font-weight: bold; font-
population/china-population/">China</td> <td style="font-weight: bold;">1,419,321,278</td> <td>-0.23 %</td> <td>-3,263,655</td>
<td>1.0</td> <td>40</td> <td>66 %</td> <td>17.39 %</td> </tr> <tr> <td>3</td> <td style="font-weight: bold; font-size:15px; text-a
population/">United States</td> <td style="font-weight: bold;">345,426,571</td> <td>0.57 %</td> <td>1,949,236</td> <td>38</td>
<td>38</td> <td>82 %</td> <td>4.23 %</td> </tr> <tr> <td>4</td> <td style="font-weight: bold; font-size:15px; text-align:left">a l
population/">Indonesia</td> <td style="font-weight: bold;">283,487,931</td> <td>0.82 %</td> <td>2,297,864</td> <td>156</td> <td>
<td>30</td> <td>59 %</td> <td>3.47 %</td> </tr> <tr> <td>5</td> <td style="font-weight: bold; font-size:15px; text-align:left">a l
```

```
[ ] cols = []
    t_header = t1.find('thead')
    t_header
```

🔗

```
<thead> <tr> <th>#</th> <th>Country (or dependency)</th> <th>Population<br> (2024)</th> <th>Yearly<br> Change</th> <th>Net<br> Change</th> <th>Density<br> (P/Km²)</th> <th>Land
Area<br> (Km²)</th> <th>Migrants<br> (net)</th> <th>Fert.<br> Rate</th> <th>Med.<br> Age</th> <th>Urban<br> Pop %</th> <th>World<br> Share</th> </tr> </thead>
```

```
[ ] for tr in t_header.find_all('tr'):
    cols.append(tr.text)
    cols
```

🔗

```
[' # Country (or dependency) Population (2024) Yearly Change Net Change Density (P/Km²) Land Area (Km²) Migrants (net) Fert. Rate Med. Age Urban Pop % World Share ']
```

```
[ ] t_body = t1.find('tbody')
t_rows = t_body.find_all('tr')
d1 = []
```

```
[ ] for rows in t_rows:
    data = []
    for row_data in rows.find_all('td'):
        data.append(row_data.text)
    d1.append(data)
d1
```

```
[[ '1',
  'India',
  '1,450,935,791',
  '0.89 %',
  '12,866,195',
  '488',
  '2,973,190',
  '-630,830',
  '2.0',
  '28',
  '37 %',
  '17.78 %'],
  ['2',
  'China',
  '1,419,321,278',
  '-0.23 %',
  '-3,263,655',
  '151',
  '9,388,211',
```

```
[ ] df = pd.DataFrame(d1)
df.head()
```

	0	1	2	3	4	5	6	7	8	9	10	11
0	1	India	1,450,935,791	0.89 %	12,866,195	488	2,973,190	-630,830	2.0	28	37 %	17.78 %
1	2	China	1,419,321,278	-0.23 %	-3,263,655	151	9,388,211	-318,992	1.0	40	66 %	17.39 %
2	3	United States	345,426,571	0.57 %	1,949,236	38	9,147,420	1,286,132	1.6	38	82 %	4.23 %
3	4	Indonesia	283,487,931	0.82 %	2,297,864	156	1,811,570	-38,469	2.1	30	59 %	3.47 %
4	5	Pakistan	251,269,164	1.52 %	3,764,669	326	770,880	-1,401,173	3.5	20	34 %	3.08 %

Practical 1-B: Perform data extraction from xml file to dataframe or CSV

```
[ ] import pandas as pd
import xml.etree.ElementTree as et
```

```
[ ] xtree = et.parse('student_data.xml')
xtree
```

 <xml.etree.ElementTree.ElementTree at 0x1dcccfdc6d0>

```
[ ] xroot = xtree.getroot()
xroot.tag
```

 'data'

```
[ ] rows = []
for node in xroot:
    sname = node.attrib.get("name") if node is not None else None
    semail = node.find("email").text if node is not None else None
    srollno = node.find('roll_no').text if node is not None else None
    sage = node.find('age').text if node is not None else None
    rows.append({'Name':sname, 'Email':semail, 'RollNo':srollno, 'Age':sage})
```

 `df = pd.DataFrame(data = rows)`
`df.head()`



	Name	Email	RollNo	Age
0	John	john@gmail.com	1	16
1	Pratap	pratap@gmail.com	2	15
2	Lydia	lydia@gmail.com	3	16
3	Subhash	shubash@gmail.com	4	17
4	Naved	naved@gmail.com	5	16

```
[ ] df.to_csv('output2.csv')
```

Practical-2 Create ML pipeline(workflow) using python libraries

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
```

```
dataset = load_breast_cancer()
```

```
[ ] df = pd.DataFrame(dataset.data, columns = dataset.feature_names)
df.head()
```



	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...

5 rows × 30 columns

```
df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
#   Column              Non-Null Count  Dtype
---  -
0   mean radius         569 non-null    float64
1   mean texture        569 non-null    float64
2   mean perimeter      569 non-null    float64
3   mean area           569 non-null    float64
4   mean smoothness     569 non-null    float64
5   mean compactness    569 non-null    float64
6   mean concavity      569 non-null    float64
```

```
[ ] df.shape
```

```
(569, 30)
```

```
[ ] df['class'] = dataset.target
```

```
[ ] logit_ml = Pipeline([('scaling', StandardScaler()),
                        ('pca', PCA(n_components = 3)),
                        ('model_logit', LogisticRegression())])
```

```
[ ] dtree_ml = Pipeline([('scaling', StandardScaler()),
                        ('pca', PCA(n_components = 3)),
                        ('model_logit', DecisionTreeClassifier())])
```

```
[ ] nb_ml = Pipeline([('scaling', StandardScaler()),
                      ('pca', PCA(n_components = 3)),
                      ('model_logit', GaussianNB())])
```

```
[ ] X = df.iloc[:, :-1]
    X.shape
```

```
(569, 30)
```

```
[ ] X.head()
```

```
(569, 30)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...

5 rows x 30 columns

```
[ ] y = df.iloc[:, -1]
    y.shape
```

```
(569,)
```

```
[ ] y.head()
```

```
(569,)
```

0	0
1	0
2	0
3	0
4	0

Name: class, dtype: int32

```
[ ] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 1) #random_state: save the testing set
```

```
[ ] my_pipeline = [logit_ml, dtree_ml, nb_ml]
```

```
[ ] pipeline_dict = {0:'Logistic_Regression', 1:'Decision_Tree', 2:'Naive_Bayes'}
```

```
[ ] for i in my_pipeline:
    i.fit(X_train, y_train)
```

```
[ ] for i, model in enumerate(my_pipeline):
    print(f"{pipeline_dict[i]}s training accuracy is: {model.score(X_train, y_train)}") #performs the model.predict step
```

```
(569,)
```

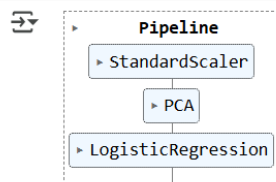
Logistic_Regression's training accuracy is: 0.9671361502347418
Decision_Tree's training accuracy is: 1.0
Naive_Bayes's training accuracy is: 0.9225352112676056

```
[ ] for i, model in enumerate(my_pipeline):
    print(f"{pipeline_dict[i]}s testing accuracy is: {model.score(X_test, y_test)}")
```

```
(569,)
```

Logistic_Regression's testing accuracy is: 0.9370629370629371
Decision_Tree's testing accuracy is: 0.9230769230769231
Naive_Bayes's testing accuracy is: 0.9090909090909091

```
[ ] logit_ml.fit(X_train, y_train)
```



```
[ ] y_pred = logit_ml.predict(X_test)
```

```
[ ] acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

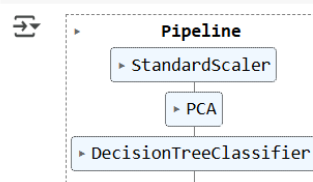
cf = classification_report(y_test, y_pred)
print("\nClassification Report:")
print(cf)
```

Accuracy: 0.9824561403508771

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.95	0.98	43
1	0.97	1.00	0.99	71
accuracy			0.98	114
macro avg	0.99	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

```
[ ] dtree_ml.fit(X_train, y_train)
```



```
[ ] y_pred_dt = dtree_ml.predict(X_test)
```

```
[ ] acc = accuracy_score(y_test, y_pred_dt)
print("Accuracy:", acc)

cf = classification_report(y_test, y_pred_dt)
print("\nClassification Report:")
print(cf)
```

Accuracy: 0.9473684210526315

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.93	0.93	43
1	0.96	0.96	0.96	71
accuracy			0.95	114
macro avg	0.94	0.94	0.94	114
weighted avg	0.95	0.95	0.95	114

Practical :3 Working with NoSQL database – HBase

Creating a LinkShare in HBase / using the existing one.

```
> hbase shell
> create 'studentrj', 'studid', 'studname', 'studaddress'
> describe 'studentrj'
```

```
cloudera@quickstart:~
File Edit View Search Terminal Help

[cloudera@quickstart ~]$ hbase shell
2025-02-02 20:22:35,277 INFO [main] Configuration.deprecation: hadoop.native.lib
is deprecated. Instead, use io.native.lib.available
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.2.0-cdh5.13.0, rUnknown, Wed Oct 4 11:16:18 PDT 2017

hbase(main):001:0> create 'studentrj','studid','studname','studaddress'
0 row(s) in 1.4990 seconds

=> Hbase::Table - studentrj
hbase(main):002:0> list
TABLE
studentrj
1 row(s) in 0.0130 seconds

=> ["studentrj"]
hbase(main):003:0> describe 'studentrj'
Table studentrj is ENABLED
studentrj
COLUMN FAMILIES DESCRIPTION
{NAME => 'studaddress', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLIC
ATION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '
0', TTL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_ME
MORY => 'false', BLOCKCACHE => 'true'}
{NAME => 'studid', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLICAT
ION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0', T
TL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_MEMORY
=> 'false', BLOCKCACHE => 'true'}
{NAME => 'studname', DATA_BLOCK_ENCODING => 'NONE', BLOOMFILTER => 'ROW', REPLIC
ATION_SCOPE => '0', VERSIONS => '1', COMPRESSION => 'NONE', MIN_VERSIONS => '0',
TTL => 'FOREVER', KEEP_DELETED_CELLS => 'FALSE', BLOCKSIZE => '65536', IN_MEMOR
Y => 'false', BLOCKCACHE => 'true'}
3 row(s) in 0.2100 seconds
```

Inserting the values in the linkshare

```
hbase(main):006:0> put 'studentrj',01 , 'studname:fname', 'Hariom'
0 row(s) in 0.1370 seconds

hbase(main):007:0> put 'studentrj',01 , 'studname:mname', 'Rajendrakumar'
0 row(s) in 0.0100 seconds

hbase(main):008:0> put 'studentrj',01 , 'studname:lname', 'Sharma'
0 row(s) in 0.0030 seconds

hbase(main):010:0> put 'studentrj',01 , 'studaddress:place', 'Amb'
0 row(s) in 0.0250 seconds

hbase(main):011:0> put 'studentrj',02 , 'studname:fname', 'Sumeet'
0 row(s) in 0.0030 seconds

hbase(main):012:0> put 'studentrj',02 , 'studname:mname', 'Neeraj'
0 row(s) in 0.0030 seconds

hbase(main):013:0> put 'studentrj',02 , 'studname:lname', 'Kulkarni'
0 row(s) in 0.0030 seconds

hbase(main):014:0> put 'studentrj',02 , 'studaddress:road', 'Devi Dayal Road'
0 row(s) in 0.0030 seconds

hbase(main):015:0> put 'studentrj',02 , 'studaddress:place', 'Mulund W'
0 row(s) in 0.0030 seconds

hbase(main):016:0> put 'studentrj',03 , 'studname:fname', 'Alka'
0 row(s) in 0.0090 seconds

hbase(main):017:0> put 'studentrj',03 , 'studname:mname', 'Deepak'
0 row(s) in 0.0030 seconds

hbase(main):018:0> put 'studentrj',03 , 'studname:lname', 'Yadav'
0 row(s) in 0.0090 seconds

hbase(main):019:0> put 'studentrj',03 , 'studaddress:road', 'PK Road'
0 row(s) in 0.0060 seconds

hbase(main):020:0> put 'studentrj',03 , 'studaddress:place', 'Thane'
0 row(s) in 0.0030 seconds
```

```

hbase(main):021:0> put 'studenttrj',03 , 'studaddress:state', 'Maharashtra'
0 row(s) in 0.0040 seconds

hbase(main):022:0> put 'studenttrj',04 , 'studname:fname', 'Amit'
0 row(s) in 0.0030 seconds

hbase(main):023:0> put 'studenttrj',04 , 'studname:mname', 'Neeraj'
0 row(s) in 0.0030 seconds

hbase(main):024:0> put 'studenttrj',04 , 'studname:lname', 'Yadav'
0 row(s) in 0.0140 seconds

hbase(main):025:0> put 'studenttrj',04 , 'studaddress:road', 'Goshala Road'
0 row(s) in 0.0060 seconds

hbase(main):026:0> put 'studenttrj',04 , 'studaddress:place', 'Mulund W'
0 row(s) in 0.0040 seconds

hbase(main):027:0> put 'studenttrj',04 , 'studaddress:state', 'Maharashtra'
0 row(s) in 0.0040 seconds

hbase(main):028:0> put 'studenttrj',05 , 'studname:fname', 'Veena'
0 row(s) in 0.0040 seconds

hbase(main):029:0> put 'studenttrj',05 , 'studname:mname', 'Pushkar'
0 row(s) in 0.0030 seconds

hbase(main):030:0> put 'studenttrj',05 , 'studname:lname', 'Mhastre'
0 row(s) in 0.0030 seconds

hbase(main):031:0> put 'studenttrj',05 , 'studaddress:road', 'JN Road'
0 row(s) in 0.0030 seconds

hbase(main):032:0> put 'studenttrj',05 , 'studaddress:place', 'Thane W'
0 row(s) in 0.0030 seconds

hbase(main):033:0> put 'studenttrj',05 , 'studaddress:state', 'Maharashtra'
0 row(s) in 0.0030 seconds

hbase(main):034:0> scan 'studenttrj'
COLUMN+CELL
ROW column=studaddress:place, timestamp=1739416370189, value=Amb
1 column=studname:fname, timestamp=1739416219908, value=Hariom
1 column=studname:lname, timestamp=1739416265370, value=Sharma
1 column=studname:mname, timestamp=1739416252638, value=Rajendrakumar
2 column=studaddress:place, timestamp=1739416479607, value=Mulund W
2 column=studaddress:road, timestamp=1739416465663, value=Devi Dayal Road
2 column=studname:fname, timestamp=1739416406214, value=Sumeet
2 column=studname:lname, timestamp=1739416429210, value=Kulkarni
2 column=studname:mname, timestamp=1739416417907, value=Neeraj
3 column=studaddress:place, timestamp=1739416627663, value=Thane
3 column=studaddress:road, timestamp=1739416605378, value=PK Road
3 column=studaddress:state, timestamp=1739416667451, value=Maharashtra
3 column=studname:fname, timestamp=1739416511444, value=Alka
3 column=studname:lname, timestamp=1739416569323, value=Yadav
3 column=studname:mname, timestamp=1739416557170, value=Deepak
4 column=studaddress:place, timestamp=1739416753449, value=Mulund W
4 column=studaddress:road, timestamp=1739416731887, value=Goshala Road
4 column=studaddress:state, timestamp=1739416761665, value=Maharashtra
4 column=studname:fname, timestamp=1739416687114, value=Amit
4 column=studname:lname, timestamp=1739416711641, value=Yadav
4 column=studname:mname, timestamp=1739416700736, value=Neeraj
5 column=studaddress:place, timestamp=1739416906086, value=Thane W
5 column=studaddress:road, timestamp=1739416889628, value=JN Road
5 column=studaddress:state, timestamp=1739416916797, value=Maharashtra
5 column=studname:fname, timestamp=1739416834840, value=Veena
5 column=studname:lname, timestamp=1739416857432, value=Mhastre
5 column=studname:mname, timestamp=1739416847454, value=Pushkar
5 row(s) in 0.0780 seconds

hbase(main):035:0> █

```

Q 1: Get the value for column place for studid 1 using column keyword

>get 'studentdata' , '1' ,{COLUMN => 'studentaddress : place' }

```

hbase(main):035:0> get 'studenttrj', 01, {COLUMN => 'studaddress:place'}
COLUMN CELL
studaddress:place timestamp=1739416370189, value=Amb
1 row(s) in 0.0130 seconds

```

Q 2: Retrieve all column values for studid 1

>get 'studentdata' , '1'

```

hbase(main):036:0> get 'studenttrj', 01
COLUMN CELL
studaddress:place timestamp=1739416370189, value=Amb
studname:fname timestamp=1739416219908, value=Hariom
studname:lname timestamp=1739416265370, value=Sharma
studname:mname timestamp=1739416252638, value=Rajendrakumar
4 row(s) in 0.0080 seconds

```

Q3: get all the record for studid 2, 3, 4

```
>get 'studentdata' , '02'  
>get 'studentdata' , '03'  
>get 'studentdata' , '04'
```

```
hbase(main):037:0> get 'studentrj', 02  
COLUMN  
studaddress:place      timestamp=1739416479607, value=Mulund W  
studaddress:road       timestamp=1739416465663, value=Devi Dayal Road  
studname:fname         timestamp=1739416406214, value=Sumeet  
studname:lname         timestamp=1739416429210, value=Kulkarni  
studname:mname         timestamp=1739416417907, value=Neeraj  
5 row(s) in 0.0090 seconds  
  
hbase(main):038:0> get 'studentrj', 03  
COLUMN  
studaddress:place      timestamp=1739416627663, value=Thane  
studaddress:road       timestamp=1739416605378, value=PK Road  
studaddress:state      timestamp=1739416667451, value=Maharashtra  
studname:fname         timestamp=1739416511444, value=Alka  
studname:lname         timestamp=1739416569323, value=Yadav  
studname:mname         timestamp=1739416557170, value=Deepak  
6 row(s) in 0.0170 seconds  
  
hbase(main):039:0> get 'studentrj', 04  
COLUMN  
studaddress:place      timestamp=1739416753449, value=Mulund W  
studaddress:road       timestamp=1739416731887, value=Goshala Road  
studaddress:state      timestamp=1739416761665, value=Maharashtra  
studname:fname         timestamp=1739416687114, value=Amit  
studname:lname         timestamp=1739416711641, value=Yadav  
studname:mname         timestamp=1739416700736, value=Neeraj  
6 row(s) in 0.0070 seconds
```

Q4: get all the column values for column family studaddress for studid 1 without using column keyword

```
>get 'studentdata' , '1' , 'studaddress'  
hbase(main):040:0> get 'studentrj', 01, 'studaddress'  
COLUMN  
studaddress:place      timestamp=1739416370189, value=Amb  
1 row(s) in 0.0150 seconds
```

Q5: change the last name for studid 1 from vase to Kelkar and check the record to see if the change are done

```
>put 'studentdata' , '1' , 'studname:lname' , 'Kelkar'  
hbase(main):041:0> put 'studentrj', 01, 'studname:lname', 'Kelkar'  
0 row(s) in 0.0100 seconds  
  
hbase(main):042:0> get 'studentrj', 01, 'studname'  
COLUMN  
studname:fname         timestamp=1739416219908, value=Harion  
studname:lname         timestamp=1739418041914, value=Kelkar  
studname:mname         timestamp=1739416252638, value=Rajendrakumar  
3 row(s) in 0.0040 seconds
```

Perform the following processes on student table

1. Count the number of records

```
>count 'studentdata'  
hbase(main):043:0> count 'studentdata'  
5 row(s) in 0.0470 seconds  
  
=> 5
```

2. Disable the table

```
>disable studentdata'  
hbase(main):044:0> disable 'studentdata'  
0 row(s) in 2.3740 seconds
```

3. Verify if the table is disabled

>is disabled 'studentdata'

```
hbase(main):046:0> is_disabled 'studentdata'
true
0 row(s) in 0.0150 seconds
```

4. Enable the table again and scan the table

>enable 'studentdata'

>scan 'studentdata'

```
hbase(main):047:0> enable 'studentdata'
0 row(s) in 1.3380 seconds

hbase(main):048:0> scan 'studentdata'
ROW COLUMN+CELL
02 column=studaddress:place, timestamp=1739416395767, value=Mulund W
02 column=studaddress:road, timestamp=1739416379203, value=Devi Dayal R
02 column=studname:fname, timestamp=1739416303249, value=Sumit
02 column=studname:lname, timestamp=1739416338217, value=Kulkarni
02 column=studname:mname, timestamp=1739416322332, value=Neeraj
03 column=studaddress:place, timestamp=1739416474139, value=Thane
03 column=studaddress:road, timestamp=1739416460740, value=PK road
03 column=studaddress:state, timestamp=1739416493475, value=Maharashtra
03 column=studname:fname, timestamp=1739416419799, value=Alka
03 column=studname:lname, timestamp=1739416437221, value=Yadav
03 column=studname:mname, timestamp=1739416428590, value=Deepak
04 column=studaddress:place, timestamp=1739416586306, value=Mulund W
04 column=studaddress:road, timestamp=1739416572708, value=Goshala Road
04 column=studaddress:state, timestamp=1739416602338, value=Maharashtra
04 column=studname:fname, timestamp=1739416515255, value=Amit
04 column=studname:lname, timestamp=1739416546543, value=Yadav
04 column=studname:mname, timestamp=1739416530513, value=Neeraj
05 column=studaddress:place, timestamp=1739416701205, value=Thane W
05 column=studaddress:road, timestamp=1739416688865, value=JN road
05 column=studaddress:state, timestamp=1739416718997, value=Maharashtra
05 column=studname:fname, timestamp=1739416622444, value=Veena
05 column=studname:lname, timestamp=1739416672272, value=Mhatre
05 column=studname:mname, timestamp=1739416635054, value=Pushkar
1 column=studaddress:place, timestamp=1739416269093, value=Dombivli
1 column=studname:fname, timestamp=1739415884674, value=Uday
1 column=studname:lname, timestamp=1739417954059, value=Kelkar
```

5. Delete the column middle name for student with id 4

>delete 'studentdata', '04', 'studname : mname'

```
hbase(main):050:0> delete 'studentdata' , '04' , 'studname:mname'
0 row(s) in 0.0520 seconds
```

6. Delete all the column families and columns for record 4

>deleteall 'studentdata', '04'

```
hbase(main):051:0> deleteall 'studentdata' , '04'
0 row(s) in 0.0190 seconds

hbase(main):052:0> scan 'studentdata'
ROW COLUMN+CELL
02 column=studaddress:place, timestamp=1739416395767, value=Mulund W
02 column=studaddress:road, timestamp=1739416379203, value=Devi Daya
02 column=studname:fname, timestamp=1739416303249, value=Sumit
02 column=studname:lname, timestamp=1739416338217, value=Kulkarni
02 column=studname:mname, timestamp=1739416322332, value=Neeraj
03 column=studaddress:place, timestamp=1739416474139, value=Thane
03 column=studaddress:road, timestamp=1739416460740, value=PK road
03 column=studaddress:state, timestamp=1739416493475, value=Maharash
03 column=studname:fname, timestamp=1739416419799, value=Alka
03 column=studname:lname, timestamp=1739416437221, value=Yadav
03 column=studname:mname, timestamp=1739416428590, value=Deepak
05 column=studaddress:place, timestamp=1739416701205, value=Thane W
05 column=studaddress:road, timestamp=1739416688865, value=JN road
05 column=studaddress:state, timestamp=1739416718997, value=Maharash
05 column=studname:fname, timestamp=1739416622444, value=Veena
05 column=studname:lname, timestamp=1739416672272, value=Mhatre
05 column=studname:mname, timestamp=1739416635054, value=Pushkar
1 column=studaddress:place, timestamp=1739416269093, value=Dombivli
1 column=studname:fname, timestamp=1739415884674, value=Uday
1 column=studname:lname, timestamp=1739417954059, value=Kelkar
1 column=studname:mname, timestamp=1739416135792, value=Brijesh
4 row(s) in 0.0240 seconds
```

7. Set the maximum number of cell changes for the student address column family to 5

```
>alter 'studentdata', NAME => 'studaddress', VERSIONS =>5
hbase(main):053:0> alter 'studentdata', NAME =>'studaddress',VERSIONS =>5
Updating all regions with the new schema...
0/1 regions updated.
1/1 regions updated.
Done.
0 row(s) in 3.0720 seconds
```

8. Delete column family student address from student table

```
>alter 'studentdata', 'delete' => 'studaddress'
hbase(main):054:0> alter 'studentdata', 'delete'=>'studaddress'
Updating all regions with the new schema...
1/1 regions updated.
Done.
0 row(s) in 2.2340 seconds
```

9. Check if the student table exists using exists and list keywords, now drop table student and again check if the student exists

```
>exists 'studentdata'
>list
>disable 'studentdata'
> drop 'studentdata'
>list
>exists 'studentdata'
hbase(main):055:0> exists 'studentdata'
Table studentdata does exist
0 row(s) in 0.0210 seconds

hbase(main):056:0> list
TABLE
studentdata
studenttrj
2 row(s) in 0.0050 seconds
hbase(main):058:0> disable 'studentdata'
0 row(s) in 2.2930 seconds

hbase(main):059:0> drop 'studentdata'
0 row(s) in 1.3220 seconds

hbase(main):060:0> list
TABLE
studenttrj
1 row(s) in 0.0050 seconds

=> ["studenttrj"]
hbase(main):061:0> exists 'studentdata'
Table studentdata does not exist
0 row(s) in 0.0200 seconds
```

Practical 4: Simulating the Datawarehouse environment

Part-1

Q1: Create a Datawarehouse database names college1

>hive

>create database college1 ;

```
[cloudera@quickstart Desktop]$ hive
```

```
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
```

```
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
```

```
hive> create database college1;
```

```
OK
```

```
Time taken: 1.44 seconds
```

Q2: check the creation of the database

>show databases ;

```
hive> show databases;
```

```
OK
```

```
college1
```

```
default
```

```
Time taken: 0.236 seconds, Fetched: 2 row(s)
```

Q3: In the table create columns of roll_no, name, course and marks, the fields in the table must be separated by ','.

> use college1 ;

> create table student (roll_no int , name string ,
Course string , marks float)

row format delimited fields terminated by ',' ;

```
hive> use college1;
```

```
OK
```

```
hive> create table student(roll_no int, name String, course String, marks float) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

```
OK
```

```
Time taken: 0.236 seconds
```

Q4: create a txt file in Hadoop cluster and load the data from the txt file in the student table

>exits ;

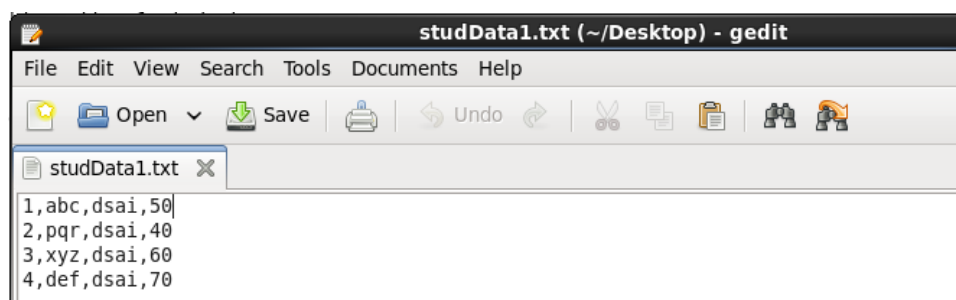
>gedit studentdata1.txt

```
hive> exit ;
```

```
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
```

```
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
```

```
[cloudera@quickstart ~]$ gedit studentdata1.txt
```



>hdfs dfs -mkdir /hadoop1

```
> hdfs dfs -mkdir /hadoop1/data
> hdfs dfs -put studentdata1.txt /hadoop1/data
> hdfs dfs -ls
```

```
[cloudera@quickstart Desktop]$ gedit studData1.txt
[cloudera@quickstart Desktop]$ gedit studData1.txt
[cloudera@quickstart Desktop]$ hdfs dfs -mkdir /hadoop
[cloudera@quickstart Desktop]$ hdfs dfs -mkdir /hadoop/data
[cloudera@quickstart Desktop]$ hdfs dfs -put studData1.txt /hadoop1/data
put: `/hadoop1/data': No such file or directory
[cloudera@quickstart Desktop]$ hdfs dfs -put studData1.txt /hadoop/data
[cloudera@quickstart Desktop]$ hdfs dfs -ls
[cloudera@quickstart Desktop]$ hdfs dfs -put studData1.txt /hadoop/data
put: `/hadoop/data/studData1.txt': File exists
```

Q5: display the data in the student table

```
>hive
> use college1;
> describe formatted student ;
> load data inpath '/hadoop1/data/studentdata1.txt' into table student ;
> select * from student ;
```

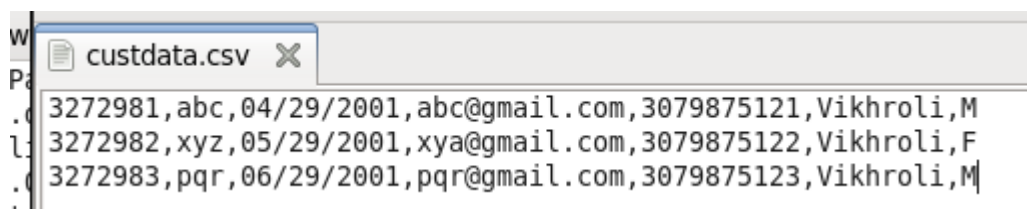
```
hive> load data inpath '/hadoop/data/studData1.txt' into table student;
Loading data to table college1.student
Table college1.student stats: [numFiles=1, totalSize=56]
OK
hive> select * from student;
OK
1      abc      dsai      50.0
2      pqr      dsai      40.0
3      xyz      dsai      60.0
4      def      dsai      70.0
Time taken: 0.293 seconds, Fetched: 4 row(s)
```

Part-2

Q1: create a csv file to store the data of insurance policy customers and enter some data

```
>gedit customer.csv
```

```
[cloudera@quickstart Desktop]$ gedit custdata.csv
```



```
W
Pa
custdata.csv
3272981,abc,04/29/2001,abc@gmail.com,3079875121,Vikhroli,M
3272982,xyz,05/29/2001,xya@gmail.com,3079875122,Vikhroli,F
3272983,pqr,06/29/2001,pqr@gmail.com,3079875123,Vikhroli,M
```

Q2: create a database name licdw and inside that create a table named customer having column id, name, dob, email-id, contact no, address and gender

```
>hive
> create database licdw;
> show databases;
> use licdw;
```

```
[cloudera@quickstart ~]$ hive
```

```
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
```

```
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
```

```
hive> create database LICDW;
```

```
OK
```

```
Time taken: 0.313 seconds
```

```
hive> show databases;
```

```
OK
```

```
collegel
```

```
default
```

```
licdw
```

```
Time taken: 0.17 seconds, Fetched: 3 row(s)
```

```
hive> use licdw;
```

```
OK
```

```
Time taken: 0.03 seconds
```

```
> create table customer ( id int,  
                           name string, dob string,  
                           email string, contactno string ,  
                           address string, gender string)  
row format delimited fields terminated by ',';
```

```
>show tables;
```

```
> describe customer;
```

```
hive> create table customer(id int, name String, date_of_birth date, email_id String, contact_no String, address String, gender String) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

```
OK
```

```
Time taken: 0.092 seconds
```

Q3: load the data from the csv file to customer table

```
>load data local inpath '/home/cloudera/customer.csv' into table customer ;
```

```
>select * from customer ;
```

```
hive> load data local inpath '/home/cloudera/Desktop/custdata.csv' into table customer;
```

```
Loading data to table licdw.customer
```

```
Table licdw.customer stats: [numFiles=1, totalSize=177]
```

```
OK
```

```
Time taken: 0.185 seconds
```

```
hive> select * from customer;
```

```
OK
```

3272981	abc	NULL	abc@gmail.com	3079875121	Vikhroli	M
3272982	xyz	NULL	xya@gmail.com	3079875122	Vikhroli	F
3272983	pqr	NULL	pqr@gmail.com	3079875123	Vikhroli	M

```
Time taken: 0.064 seconds, Fetched: 3 row(s)
```

Practical 5 A: Working with different types of tables in hive

Internal Table:

```
>hive
> create database licdw;
> show databases;
> use licdw;
> create table customer ( id int,
                        name string, dob string,
                        email string, contactno string ,
                        address string, gender string)
row format delimited fields terminated by ',';
>show tables ;
> describe customer;
```

```
cloudera@quickstart:~
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.p
roperties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> create database licdw;
OK
Time taken: 1.855 seconds
hive> use licdw;
OK
Time taken: 0.077 seconds
hive> create table customer(id int,
> name string,
> dob string,
> contact string,
> address string,
> gender string)row format delimited
> fields terminated by ',';
OK
Time taken: 0.431 seconds
hive> describe customer;
OK
id                int
name              string
dob              string
contact          string
address          string
gender           string
Time taken: 0.259 seconds, Fetched: 6 row(s)
hive> describe formatted student;
FAILED: SemanticException [Error 10001]: Table not found student
hive> exit;
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
[cloudera@quickstart ~]$ gedit customer.csv
[cloudera@quickstart ~]$ gedit policydetails.csv
```

>load data local inpath '/home/cloudera/customer.csv' into table customer ;

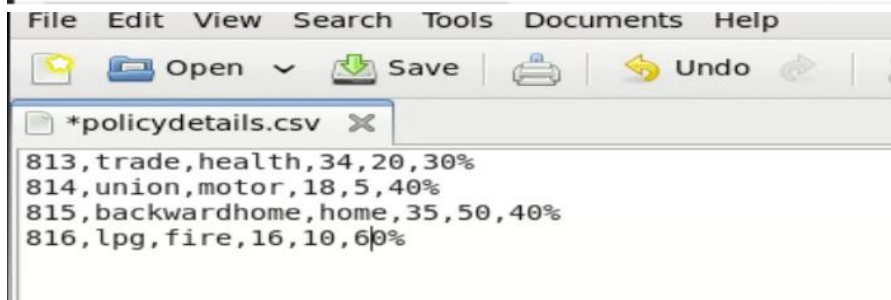
>select * form customer ;

```
hive> load data local inpath '/home/cloudera/customer.csv' into table customer;
Loading data to table licdw.customer
Table licdw.customer stats: [numFiles=1, totalSize=140]
OK
Time taken: 0.618 seconds
hive> select * from customer;
OK
1      abc      04/09/2001      123456 vikhroli      M
2      abcd     05/09/2001      98123456      vasai      M
3      xyz      04/03/2005      12345678      virar      F
4      pqr      05/09/2000      3523456 vikhroli      M
Time taken: 0.507 seconds, Fetched: 4 row(s)
hive>
```

External Table:

```
>gedit policydetails.csv
```

```
[cloudera@quickstart ~]$ gedit customer.csv  
[cloudera@quickstart ~]$ gedit policydetails.csv
```



```
>create external table policy_details (id int ,  
    name string,  
    type string,  
    age_criteria int,  
    tenure int ,  
    maturity string) row format delimited fields terminated by ','  
    stored as textfile location '/home/cloudera/policy_detail_data';
```

```
>load data local inpath '/home/cloudera/Desktop/policydetails.csv' into table policy_details;
```

```
> select * from policy_details ;
```

```
hive> create external table policy_details(id int,  
    > name string,  
    > type string,  
    > age_criteria int,  
    > tenure int,  
    > maturity string)  
    > row format delimited fields terminated by ','  
    > stored as textfile location '/home/cloudera/policy_detail_data';  
OK  
Time taken: 0.099 seconds
```

```
hive> load data local inpath '/home/cloudera/policydetails.csv' overwrite into table policy_details;  
ls;  
Loading data to table lidw.policy_details  
Table lidw.policy_details stats: [numFiles=1, numRows=0, totalSize=107, rawDataSize=0]  
OK  
Time taken: 0.267 seconds  
hive> select * from policy_details;  
OK  
813      trade      health    34        20        30%  
814      union      motor     18         5        40%  
815      backwardhome  home     35        50        40%  
816      lpg        fire      16         10       60%  
Time taken: 0.053 seconds, Fetched: 4 row(s)
```

```
>hdfs dfs -ls hdfs://quickstart.cloudera:8020/home/cloudera/policy_detail_data
```

```
> hdfs dfs -cat hdfs://quickstart.cloudera:8020/home/cloudera/policy_detail_data/policydetails.csv
```

```
hive> select * from policy_details;
OK
813      trade      health  34      20      30%
814      union      motor   18      5       40%
815      backwardhome  home    35      50      40%
816      lpg        fire    16      10      60%
Time taken: 0.437 seconds, Fetched: 4 row(s)
hive> exit;
WARN: The method class org.apache.commons.logging.impl.SLF4JLogFactory#release() was invoked.
WARN: Please see http://www.slf4j.org/codes.html#release for an explanation.
[cloudera@quickstart ~]$ hdfs dfs -ls hdfs://quickstart.cloudera:8020/home/cloudera/policy_detail
_data1
Found 1 items
-rwxr-xr-x  1 cloudera supergroup      107 2025-02-19 19:46 hdfs://quickstart.cloudera:8020/home/cloudera/policy_detail_data1/policydetails.csv
[cloudera@quickstart ~]$ hdfs dfs -cat hdfs://quickstart.cloudera:8020/home/cloudera/policy_detail_data1/policydetails.csv
813,trade,health,34,20,30%
814,union,motor,18,5,40%
815,backwardhome,home,35,50,40%
816,lpg,fire,16,10,60%
[cloudera@quickstart ~]$
```

Temp Table :

>create temporary table policy_details_temp (id int,

name string,

type string,

age_criteria int ,

tenure int,

maturity string)

> describe policy_details_temp ;

> insert into policy_details_temp values (32, 'abc', 'h', 32 , 3 , '40%');

```
hive> create temporary table policy_detail_temp(id int,
> name string,
> type string,
> age_criteria int,
> tenure int,
> maturity string);
OK
Time taken: 0.033 seconds
hive> describe policy_detail_temp;
OK
id                int
name              string
type              string
age_criteria      int
tenure            int
maturity          string
Time taken: 0.014 seconds, Fetched: 6 row(s)
hive> describe policy_detail;
FAILED: SemanticException [Error 10001]: Table not found policy_detail
hive> describe policy_details;
OK
id                int
name              string
type              string
age_criteria      int
tenure            int
maturity          string
Time taken: 0.095 seconds, Fetched: 6 row(s)
hive> insert into policy_detail_temp values(32,'t','h',24,3,'40%');
Query ID = cloudera_20250219195353_3ee8af51-db3b-4208-8b8c-15a8a648fa2b
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740020798847_0001, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1740020798847_0001/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1740020798847_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2025-02-19 19:53:44,816 Stage-1 map = 0%, reduce = 0%
2025-02-19 19:53:52,632 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.55 sec
MapReduce Total cumulative CPU time: 1 seconds 550 msec
Ended Job = job_1740020798847_0001
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/tmp/hive/cloudera/f832a9ca-dcad-4c6a-8527-a7ebda8dc7ad/ tmp space.db/dfd38a93-7cd7-47fa-8b7e-91b956327ccd/.hive-staging_hive_2025-02-19_19-53-32_782_907012048806482679-1/-ext-10000
Loading data to table lidcw.policy_detail_temp
Table lidcw.policy_detail_temp stats: [numFiles=1, numRows=1, totalSize=16, rawDataSize=15]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 1.55 sec HDFS Read: 4973 HDFS Write: 96 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 550 msec
OK
Time taken: 21.037 seconds
hive>
```

```
hive> insert into policy_detail_temp values(33,'t','m',15,4,'30%');
```

>select * from policy_details_temp;

> show tables ;

```
Time taken: 0.035 seconds, Fetched: 2 row(s)
hive> select * from policy_detail_temp;
OK
32      t      h      24      3      40%
33      t      m      15      4      30%
Time taken: 0.035 seconds, Fetched: 2 row(s)
hive> █
```

Close the existing hive session and open a new hive session

>use licdw;

>show tables

```
[cloudera@quickstart ~]$ hive;
Logging initialized using configuration in file:/opt/hive/conf/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to HUE is recommended.
hive> use licdw;
OK
Time taken: 0.389 seconds
hive> show tables;
OK
customer
policy_details
Time taken: 0.239 seconds, Fetched: 2 row(s)
hive> █
```

Practical 5 B: Demonstrating table partitioning and clustering (bucketing in hive)

```
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.p
roperties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> create database db2 ;
OK
Time taken: 0.852 seconds
hive> show databases;
OK
db2
default
licdw
Time taken: 0.341 seconds, Fetched: 3 row(s)
hive> use db2;
OK
```

Q1: create and partition a table named emp_partition on the mobile_no column and load data into it

- > create table emppartition(emp_id int ,
name string) partitioned by (m_no string)
row format delimited fields terminated by ',' ;
- > set hive.exec.dynamic.partition.mode = nonstrict;
- > insert into emppartition partition(m_no) values (1, 'sam' , '123455667') ;

```
hive> create table emppartition(emp_id int,name string) partitioned by(m_no string)
> row format delimited fields terminated by ',';
OK
Time taken: 0.174 seconds
hive>
```

```
hive> create table emppartition(emp_id int,name string) partitioned by(m_no string)
> row format delimited fields terminated by ',';
OK
Time taken: 0.174 seconds
hive> set hive.exec.dynamic.partition.mode=nonstrict;
hive> insert into emppartition partition(m_no) values(1,'sam','123455667');
Query ID = cloudera_20250220191212_9f6b0bde-2b15-497e-8261-794bf71c19e3
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1740105280171_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1740105280171_0002
```

```
hive> select * from emppartition;
OK
1      sam      123455667
Time taken: 0.106 seconds, Fetched: 1 row(s)
hive> insert into emppartition partition(m_no) values(2,'sam2','123455667');
Query ID = cloudera_20250220191313_a44864f1-c83b-4fb1-a96c-dd59e38c4f65
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1740105280171_0003/
```

```
hive> select * from emppartition;
OK
1      sam      123455667
2      sam2     123455667
Time taken: 0.111 seconds, Fetched: 2 row(s)
hive> insert into emppartition partition(m_no) values(3,'ravi','923455667');
Query ID = cloudera_20250220191313_8d87124f-b47e-4408-8c42-9571bfceald5
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0004, Tracking URL = http://quickstart.cloudera:8088,
cation_1740105280171_0004/
```

```
hive> insert into emppartition partition(m_no) values(4,'ravi2','923455667');
Query ID = cloudera_20250220191515_0b1b67e8-02ba-4f1d-9fe6-205c4f73fc94
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0005, Tracking URL = http://quickstart.cloude
cation_1740105280171_0005/
```

```

hive> select * from emppartition;
OK
1      sam      123455667
2      sam2     123455667
3      ravi     923455667
4      ravi2    923455667
Time taken: 0.104 seconds, Fetched: 4 row(s)

```

Q2: create a table empbucknopartition with only bucketing on the mobile no. column load data into it

```

> create table emp_buck_no_partition( emp_id int , name string , m_no string)
  clustered by(m_no) into 5 buckets ;
> insert into emp_buck_no_partition values(1 , 'xyz' , '12344555') ;
> select * from emp_buck_no_partition ;

```

```

hive> create table emp_buck_no_partition(emp_id int,name string,m_no string) clustered by(m_no) i
nto 5 buckets;
OK
Time taken: 0.085 seconds
hive> insert into emp_buck_no_partition values (1,'xyz',12344555);
Query ID = cloudera_20250220192121_b648d4b9-b059-47cb-8141-84cef8087ea1
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1740105280171_0006/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1740105280171_0006

```

```

hive> select * from emp_buck_no_partition;
OK
1      xyz      12344555
Time taken: 0.046 seconds, Fetched: 1 row(s)

```

Q3: Create a table named emp_buck_with_partition on m_no2. column and load data into it

```

> create table emp_buck_with_partition ( emp_id int , name string , m_no string)
  partitioned by (m_no2 string) clustered by(m_no) into 5 buckets ;
> set hive.exec.dynamic.partition.mode = nonstrict ;
> insert into emp_buck_with_partition partition(m_no2) values(1, 'abc' , '2345667' , '1234455' );

```

```

hive> create table emp_buck_with_partition(emp_id int,name string,m_no string) partitioned by(m_n
o2 string) clustered by(m_no) into 5 buckets;
OK
Time taken: 0.092 seconds
hive> set hive.exec.dynamic.partition.mode=nonstrict;
hive> insert into emp_buck_with_patition(1,'abc','2345667','1234455');
NoViableAltException(296@[])
    at org.apache.hadoop.hive.ql.parse.HiveParser_IdentifierParser.identifier(HiveParser_Ide
ntifiersParser.java:11566)
    at org.apache.hadoop.hive.ql.parse.HiveParser.identifier(HiveParser.java:44913)
    at org.apache.hadoop.hive.ql.parse.HiveParser.columnName(HiveParser.java:37161)

```

```

hive> insert into emp_buck_with_partition partition(m_no2) values (1,'abc','2345667','1234455');
Query ID = cloudera_20250220192929_b594fbd9-451b-44cc-9e54-4a2581dca464
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1740105280171_0007, Tracking URL = http://quickstart.cloudera:8088/proxy/appli
cation_1740105280171_0007/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1740105280171_0007

```

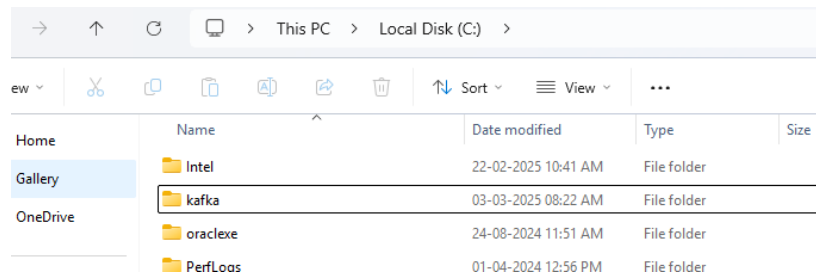
```

hive> select * from emp_buck_with_partition;
OK
1      abc      2345667 1234455
Time taken: 0.07 seconds, Fetched: 1 row(s)

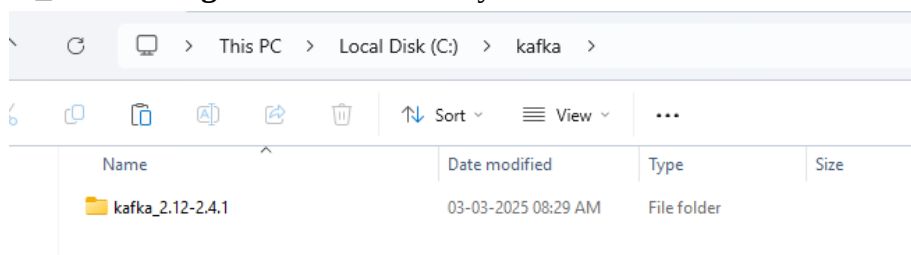
```

Practical 6: Demonstrating Publisher/Subscriber messaging system using Kafka

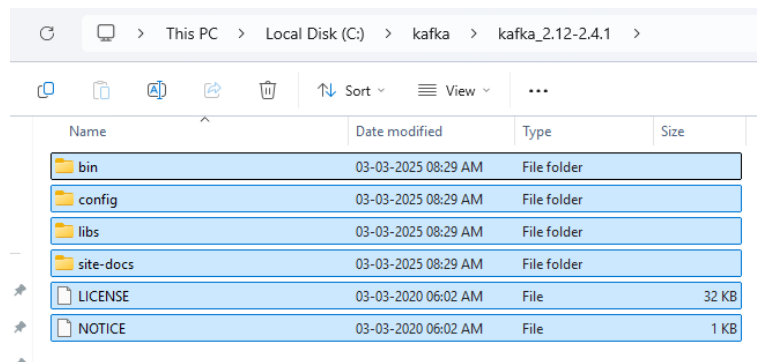
Next the setup for Kafka: first create a folder named **Kafka** in your C drive



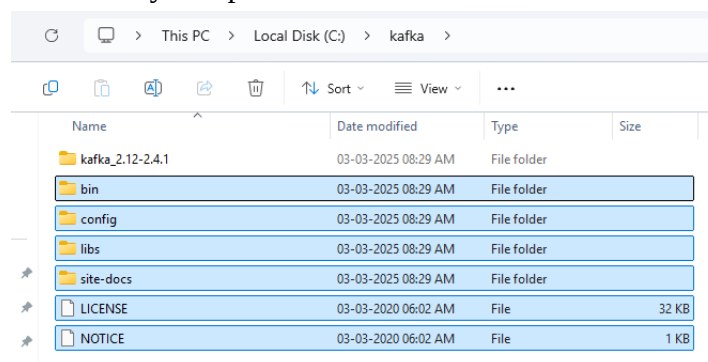
And unzip the **kafka_2.12-2.4.1.tgz** files in to the newly created kafka folder in c drive



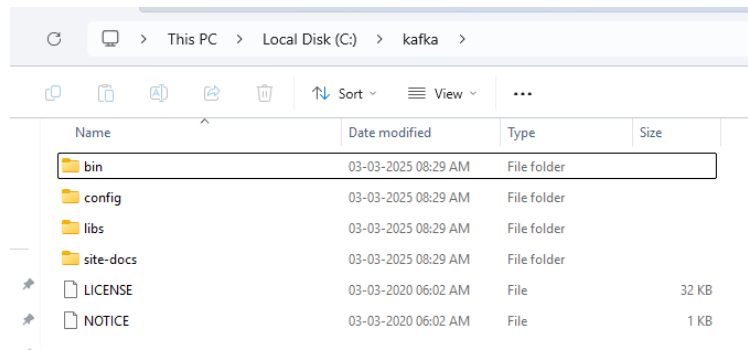
Open the **folder kafka_2.12-2.4.1** and select all the files and click on cut



Then go back to the **c:\kafka** directory and paste the files



At last delete the delete the **kafka_2.12-2.4.1** folder as it will be empty since we removed the data from it



Next click on the **config** folder , here you will find two properties files named server.properties and zookeeper.properties

Name	Date modified	Type	Size
bin	03-03-2025 08:29 AM	File folder	
config	03-03-2025 08:29 AM	File folder	
libs	03-03-2025 08:29 AM	File folder	
site-docs	03-03-2025 08:29 AM	File folder	
LICENSE	03-03-2020 06:02 AM	File	32 KB
NOTICE	03-03-2020 06:02 AM	File	1 KB

producer	03-03-2020 06:02 AM	Properties Source ...	2 KB
server	03-03-2020 06:02 AM	Properties Source ...	7 KB
tools-log4j	03-03-2020 06:02 AM	Properties Source ...	2 KB
trogdor.conf	03-03-2020 06:02 AM	CONF File	2 KB
zookeeper	03-03-2020 06:02 AM	Properties Source ...	2 KB

Open the **zookeeper.properties** by any text editor on line number 16 you will see a **dataDir=/tmp/zookeeper**. We need to replace the **/tmp** with our kafka path that is **C:\kafka** and after the **zookeeper put -data** , save and exit

So it will look like the below image :

```
14 # limitations under the License.
15 # the directory where the snapshot is stored.
16 dataDir=C:\kafka\zookeeper-data
17 # the port at which the clients will connect
18 clientPort=2181
```

Similarly open the **server.properties** file on line no 60 do the changes on **logs.dirs** as show below in image : Save and exit after the changes

```
58
59 # A comma separated list of directories under which to store log files
60 logs.dirs=C:\kafka\kafka-logs
61
62 # The default number of log partitions per topic. More partitions allow
63 # parallelism for consumption, but this will also result in more files
```

Next open the power shell and change the directory to the kafka folder in C drive :

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\User35> cd C:\kafka
PS C:\kafka> |
```

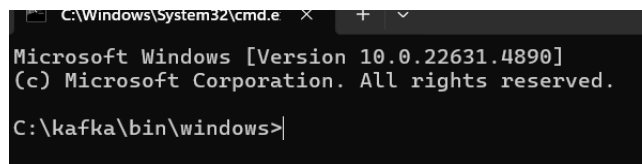
Next write the command `.\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties` and click on enter

```
PS C:\Users\User35> cd C:\kafka
PS C:\kafka> .\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties
[2025-03-03 08:40:33,079] INFO Reading configuration from: .\config\zookeeper.properties (org.apache.zoo
keeper.server.quorum.QuorumPeerConfig)
[2025-03-03 08:40:33,087] WARN C:kafkazookeeper-data is relative. Prepend .\ to indicate that you're su
re! (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-03-03 08:40:33,091] INFO clientPortAddress is 0.0.0.0:2181 (org.apache.zookeeper.server.quorum.Qu
orumPeerConfig)
```

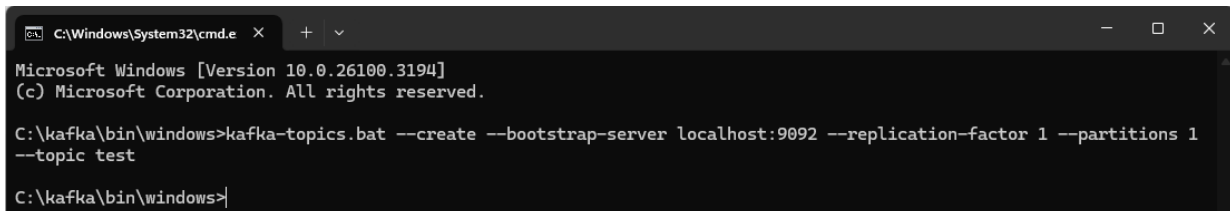
Now minimize this powershell (Do not click on cut) and open a new power shell with same path of C:\kafka
And write the command: `.\bin\windows\kafka-server-start.bat .\config\server.properties` and enter

```
PS C:\Users\User35> cd C:\kafka
PS C:\kafka> .\bin\windows\kafka-server-start.bat .\config\server.properties
[2025-03-03 08:43:44,213] INFO Registered kafka:type=kafka.Log4jController MBean (kafka.utils.Log4jControllerRegistratio
n$)
[2025-03-03 08:43:44,558] INFO starting (kafka.server.KafkaServer)
[2025-03-03 08:43:44,558] INFO Connecting to zookeeper on localhost:2181 (kafka.server.KafkaServer)
[2025-03-03 08:43:44,574] INFO [ZooKeeperClient Kafka server] Initializing a new session to localhost:2181. (kafka.zooke
eper.ZooKeeperClient)
```

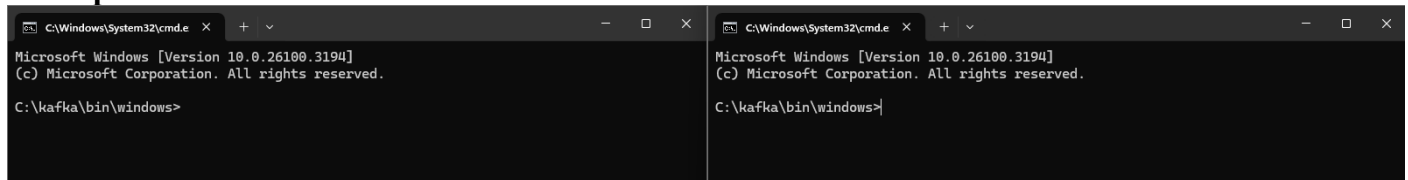
Now while this 2 powershells are running in the background go the **C drive kafka folder in that \bin in that \windows and open a cmd from that**



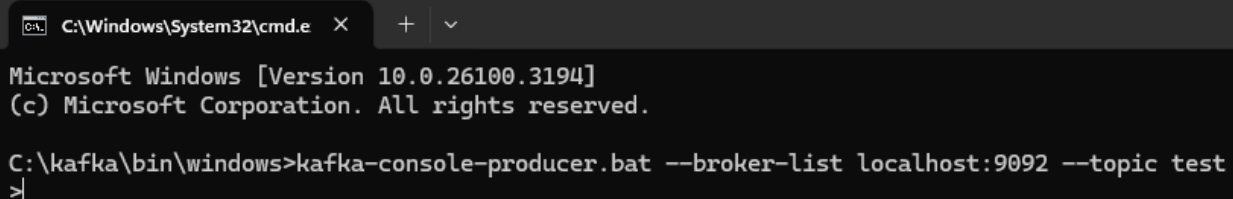
And run the command `kafka-topics.bat --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic test`



Now **open two new cmd** with same location as the `c:\kafka\bin\windows` :



In one cmd type the command : `kafka-console-producer.bat --broker-list localhost:9092 --topic test`



And in the second cmd type the command : **kafka-console-consumer.bat --bootstrap-server localhost:9092--topic test --from-beginning**

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. All rights reserved.

C:\kafka\bin\windows>kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic test --from-beginning
|
```

Now if we pass some string form the first cmd that has > (> Kafka practical) it will display the data in the 2nd cmd :

<pre>C:\Windows\System32\cmd.e X + v Microsoft Windows [Version 10.0.26100.3194] (c) Microsoft Corporation. All rights reserved. C:\kafka\bin\windows>kafka-console-producer.bat --broker-list localhost:9092 --topic test >kafka practical >{Name : 'Hari'; age : 21} ></pre>	<pre>C:\Windows\System32\cmd.e X + v Microsoft Windows [Version 10.0.26100.3194] (c) Microsoft Corporation. All rights reserved. C:\kafka\bin\windows>kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic test --from-beginning kafka practical {Name : 'Hari'; age : 21} </pre>
---	--

Practical 7: Write a simple Pyspark driver program

```
[ ] pip install pyspark
```

 Requirement already satisfied: pyspark in /usr/local/lib/python3.11/dist-packages (3.5.5)
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11/dist-packages (from pyspark) (0.10.9.7)

```
[ ] from pyspark.sql import SparkSession
```

```
[ ] spark = SparkSession.builder.appName('myapp').getOrCreate()
```

```
[ ] sc = spark.sparkContext
```

```
[ ] rdd = sc.parallelize([1,2,3,4,5])
```

```
[ ] rdd_squared = rdd.map(lambda x: x*x)
```

```
[ ] rdd_squared.collect()
```

 [1, 4, 9, 16, 25]

```
[ ] spark.stop()
```

- In pyspark the driver program and spark context play an important role
- Spark context is the entry point to spark's core functionalities before spark 2.0 it was used directly, but after spark 2.0 a higher level api sparksession is the new entry point that internally contains spark context.
- In the above code when we run the script the driver program starts and creates spark Context it builds a DAG of transformations but when an action in this case collect is called spark executes the transformations.

Practical 7 B: Working with different transformations and actions on RDD by fetching data from an external file

```
pip install pyspark==3.5.0
```

```
Requirement already satisfied: pyspark==3.5.0 in /usr/local/lib/python3.11/dist-packages (3.5.0)  
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11/dist-packages (from pyspark==3.5.0) (0.10.9.7)
```

```
[ ] from pyspark.context import SparkContext
```

```
[ ] #sc.stop()
```

```
[ ] sc = SparkContext('local', 'iris')  
iris1 = sc.textFile('iris.csv', 2)  
iris1.collect()
```

```
["", "Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species",  
"1", 5.1, 3.5, 1.4, 0.2, "setosa",  
"2", 4.9, 3.1, 4.0, 0.2, "setosa",  
"3", 4.7, 3.2, 1.3, 0.2, "setosa",  
"4", 4.6, 3.1, 1.5, 0.2, "setosa",  
"5", 5.3, 6.1, 4.0, 0.2, "setosa",  
"6", 5.4, 3.9, 1.7, 0.4, "setosa",  
"7", 4.6, 3.4, 1.4, 0.3, "setosa",  
"8", 5.3, 3.4, 1.5, 0.2, "setosa",  
"9", 4.4, 2.9, 1.4, 0.2, "setosa",  
"10", 4.9, 3.1, 1.5, 0.1, "setosa",
```

```
[ ] partitioned_data = iris1.glom().collect()  
for i, partition in enumerate(partitioned_data):  
    print(f"Partition-{i+1} has {len(partition)} records")
```

```
Partition-1 has 78 records  
Partition-2 has 73 records
```

```
[ ] iris2 = sc.parallelize(iris1.collect(), 8)  
iris2.getNumPartitions()
```

```
8
```

```
for i in iris2.collect():  
    print(i)
```

```
"12", 4.8, 3.4, 1.6, 0.2, "setosa"  
"13", 4.8, 3.1, 4.0, 0.1, "setosa"  
"14", 4.3, 3.1, 1.1, 0.1, "setosa"  
"15", 5.8, 4.1, 2.0, 0.2, "setosa"  
"16", 5.7, 4.4, 1.5, 0.4, "setosa"  
"17", 5.4, 3.9, 1.3, 0.4, "setosa"
```

```
[ ] header = iris2.first()  
iris2 = iris2.filter(lambda x: x != header)  
iris2.collect()
```

```
["1", 5.1, 3.5, 1.4, 0.2, "setosa",  
"2", 4.9, 3.1, 4.0, 0.2, "setosa",  
"3", 4.7, 3.2, 1.3, 0.2, "setosa",  
"4", 4.6, 3.1, 1.5, 0.2, "setosa",  
"5", 5.3, 6.1, 4.0, 0.2, "setosa",  
"6", 5.4, 3.9, 1.7, 0.4, "setosa",  
"7", 4.6, 3.4, 1.4, 0.3, "setosa",
```

```
[ ] iris2.count()
```

```
150
```

```
[ ] iris3 = iris2.map(lambda x: x*2)  
iris3.collect()
```

```
["1", 5.1, 3.5, 1.4, 0.2, "setosa" "1", 5.1, 3.5, 1.4, 0.2, "setosa",  
"2", 4.9, 3.1, 4.0, 0.2, "setosa" "2", 4.9, 3.1, 4.0, 0.2, "setosa",  
"3", 4.7, 3.2, 1.3, 0.2, "setosa" "3", 4.7, 3.2, 1.3, 0.2, "setosa",  
"4", 4.6, 3.1, 1.5, 0.2, "setosa" "4", 4.6, 3.1, 1.5, 0.2, "setosa",  
"5", 5.3, 6.1, 4.0, 0.2, "setosa" "5", 5.3, 6.1, 4.0, 0.2, "setosa",  
"6", 5.4, 3.9, 1.7, 0.4, "setosa" "6", 5.4, 3.9, 1.7, 0.4, "setosa",  
"7", 4.6, 3.4, 1.4, 0.3, "setosa" "7", 4.6, 3.4, 1.4, 0.3, "setosa",  
"8", 5.3, 3.4, 1.5, 0.2, "setosa" "8", 5.3, 3.4, 1.5, 0.2, "setosa",  
"9", 4.4, 2.9, 1.4, 0.2, "setosa" "9", 4.4, 2.9, 1.4, 0.2, "setosa",  
"10", 4.9, 3.1, 1.5, 0.1, "setosa" "10", 4.9, 3.1, 1.5, 0.1, "setosa",
```

```
[ ] iris4 = iris2.flatMap(lambda x: x.split(","))
iris4.collect()
```

```
⇒ ['"1"',
    '5.1',
    '3.5',
    '1.4',
    '0.2',
    '"setosa"',
    '"2"',
    '4.9',
    '3',
    '1.4',
    '0.2',
    '"setosa"',
    '"3"',
    '4.7',
    '3.2',
    '1.3',
    '0.2',
```

```
[ ] partitions = iris4.glom().collect()
print("Total partitions:", len(partitions))
```

```
⇒ Total partitions: 8
```

```
[ ] partitions[0]
```

```
⇒ ['"1"',
    '5.1',
    '3.5',
    '1.4',
    '0.2',
    '"setosa"',
    '"2"',
    '4.9',
    '3',
    '1.4',
    '0.2',
    '"setosa"',
```

```
[ ] iris5 = iris4.map(lambda x: x*2)
```

```
[ ] iris4.take(5)
```

```
⇒ ['"1"', '5.1', '3.5', '1.4', '0.2']
```

```
▶ iris6 = iris1.flatMap(lambda x: x.split(",")).filter(lambda x: x=="virginica").map(lambda x: (x,1))
partitions = iris6.glom().collect()
partitions
```

```
⇒ [[],
    [('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
     ('"virginica"', 1),
```

```
[ ] species = ["virginica", "setosa", "versicolor"]
species1 = iris1.flatMap(lambda x: x.split(",")).filter(lambda x: x in species)
species1.collect()
```

```
⇒ ["setosa",
    "setosa",
    "setosa",
    "setosa",
    "setosa",
    "setosa",
    "setosa",
    "setosa",
    "setosa",
```

```
[ ] species2 = species1.groupBy(lambda x: x).mapValues(lambda x: len(x))
species2.collect()
```

```
⇒ [('"setosa"', 50), ('"versicolor"', 50), ('"virginica"', 50)]
```

Practical 8: Data Exploration using Spark SQL

```
!pip install pyspark==3.5.5
```

```
Collecting pyspark==3.5.5
  Using cached pyspark-3.5.5-py2.py3-none-any.whl
Collecting py4j==0.10.9.7 (from pyspark==3.5.5)
  Using cached py4j-0.10.9.7-py2.py3-none-any.whl.metadata (1.5 kB)
Using cached py4j-0.10.9.7-py2.py3-none-any.whl (200 kB)
Installing collected packages: py4j, pyspark
  Attempting uninstall: py4j
    Found existing installation: py4j 0.10.7
    Uninstalling py4j-0.10.7:
      Successfully uninstalled py4j-0.10.7
  Attempting uninstall: pyspark
    Found existing installation: pyspark 2.4.8
    Uninstalling pyspark-2.4.8:
      Successfully uninstalled pyspark-2.4.8
Successfully installed py4j-0.10.9.7 pyspark-3.5.5
WARNING: Ignoring invalid distribution ~yspark (C:\Users\uday8\AppData\Local\Programs\Python\Python311\Lib\site-packages)
WARNING: Ignoring invalid distribution ~yspark (C:\Users\uday8\AppData\Local\Programs\Python\Python311\Lib\site-packages)
WARNING: Ignoring invalid distribution ~yspark (C:\Users\uday8\AppData\Local\Programs\Python\Python311\Lib\site-packages)

[notice] A new release of pip is available: 24.3.1 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
[ ] from pyspark.sql import SparkSession
```

```
[ ] spark = SparkSession.builder.appName('DataProcessingRDD').getOrCreate()
```

```
[ ] df_iris = spark.read.option('header', 'true').csv('iris.csv', inferSchema=True)
```

```
[ ] df_iris.show()
```

```
+---+-----+-----+-----+-----+-----+
|_c0|Sepal.Length|Sepal.Width|Petal.Length|Petal.Width|Species|
+---+-----+-----+-----+-----+-----+
| 1|      5.1|      3.5|      1.4|      0.2| setosa|
| 2|      4.9|      3.0|      1.4|      0.2| setosa|
| 3|      4.7|      3.2|      1.3|      0.2| setosa|
| 4|      4.6|      3.1|      1.5|      0.2| setosa|
| 5|      5.0|      3.6|      1.4|      0.2| setosa|
| 6|      5.4|      3.9|      1.7|      0.4| setosa|
| 7|      4.6|      3.4|      1.4|      0.3| setosa|
| 8|      5.0|      3.4|      1.5|      0.2| setosa|
| 9|      4.4|      2.9|      1.4|      0.2| setosa|
|10|      4.9|      3.1|      1.5|      0.1| setosa|
|11|      5.4|      3.7|      1.5|      0.2| setosa|
```

```
[ ] # renaming columns
df_iris = df_iris.toDF('ID', 'Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width', 'Species')
```

```
[ ] df_iris.show()
```

```
+---+-----+-----+-----+-----+-----+
| ID|Sepal_Length|Sepal_Width|Petal_Length|Petal_Width|Species|
+---+-----+-----+-----+-----+-----+
| 1|      5.1|      3.5|      1.4|      0.2| setosa|
| 2|      4.9|      3.0|      1.4|      0.2| setosa|
| 3|      4.7|      3.2|      1.3|      0.2| setosa|
| 4|      4.6|      3.1|      1.5|      0.2| setosa|
| 5|      5.0|      3.6|      1.4|      0.2| setosa|
| 6|      5.4|      3.9|      1.7|      0.4| setosa|
| 7|      4.6|      3.4|      1.4|      0.3| setosa|
| 8|      5.0|      3.4|      1.5|      0.2| setosa|
| 9|      4.4|      2.9|      1.4|      0.2| setosa|
```

```
[ ] df_iris.count()
```

```
150
```

```
[ ] df_iris.summary().show()
```

```
+---+-----+-----+-----+-----+-----+
|summary|          ID|      Sepal_Length|      Sepal_Width|      Petal_Length|      Petal_width|      Species|
+---+-----+-----+-----+-----+-----+
| count|         150|          150|          150|          150|          150| |
| mean|       75.5| 5.843333333333335| 3.057333333333334| 3.758000000000027| 1.199333333333334| NULL|
| stddev|43.445367992456916|0.8280661279778637|0.43586628493669793|1.7652982332594662|0.7622376689603467| NULL|
| min|         1|         4.3|         2.0|         1.0|         0.1| setosa|
| 25%|        38|         5.1|         2.8|         1.6|         0.3| NULL|
| 50%|        75|         5.8|         3.0|         4.3|         1.3| NULL|
| 75%|       113|         6.4|         3.3|         5.1|         1.8| NULL|
| max|       150|         7.9|         4.4|         6.9|         2.5| virginica|
```

```
[ ] df_iris.select('Sepal_Length').summary().show()
```

```
+-----+-----+
|summary|   Sepal_Length|
+-----+-----+
|  count|             150|
|   mean|  5.843333333333335|
| stddev| 0.8280661279778637|
|    min|              4.3|
|   25%|              5.1|
|   50%|              5.8|
|   75%|              6.4|
|    max|              7.9|
+-----+-----+
```

```
[ ] df_iris.select('Sepal_Length','Petal_Length').show()
```

```
+-----+-----+
|Sepal_Length|Petal_Length|
+-----+-----+
|          5.1|          1.4|
|          4.9|          1.4|
|          4.7|          1.3|
|          4.6|          1.5|
|          5.0|          1.4|
|          5.4|          1.7|
|          4.6|          1.4|
|          5.0|          1.5|
|          4.4|          1.4|
```

```
[ ] df_iris.createOrReplaceTempView('iristab')
```

```
[ ] df_iris1 = spark.sql("""select * from iristab where Petal_Length <2.5""")
```

```
[ ] df_iris1.show()
```

```
+--+-----+-----+-----+-----+-----+
| ID|Sepal_Length|Sepal_Width|Petal_Length|Petal_Width|Species|
+--+-----+-----+-----+-----+-----+
|  1|          5.1|          3.5|          1.4|          0.2| setosa|
|  2|          4.9|          3.0|          1.4|          0.2| setosa|
|  3|          4.7|          3.2|          1.3|          0.2| setosa|
|  4|          4.6|          3.1|          1.5|          0.2| setosa|
|  5|          5.0|          3.6|          1.4|          0.2| setosa|
|  6|          5.4|          3.9|          1.7|          0.4| setosa|
|  7|          4.6|          3.4|          1.4|          0.3| setosa|
|  8|          5.0|          3.4|          1.5|          0.2| setosa|
|  9|          4.4|          2.9|          1.4|          0.2| setosa|
| 10|          4.9|          3.1|          1.5|          0.1| setosa|
| 11|          5.4|          3.7|          1.5|          0.2| setosa|
| 12|          4.8|          3.4|          1.6|          0.2| setosa|
| 13|          4.8|          3.0|          1.4|          0.1| setosa|
```

```
[ ] df_iris1.count()
```

```
50
```

```
df_iris2 = spark.sql(""" select * from iristab where Petal_Length between 2.5 and 4.5""")
df_iris2.show()
```

```
+--+-----+-----+-----+-----+-----+
| ID|Sepal_Length|Sepal_Width|Petal_Length|Petal_Width|Species|
+--+-----+-----+-----+-----+-----+
| 52|          6.4|          3.2|          4.5|          1.5|versicolor|
| 54|          5.5|          2.3|          4.0|          1.3|versicolor|
| 56|          5.7|          2.8|          4.5|          1.3|versicolor|
| 58|          4.9|          2.4|          3.3|          1.0|versicolor|
| 60|          5.2|          2.7|          3.9|          1.4|versicolor|
| 61|          5.0|          2.0|          3.5|          1.0|versicolor|
| 62|          5.9|          3.0|          4.2|          1.5|versicolor|
| 63|          6.0|          2.2|          4.0|          1.0|versicolor|
| 65|          5.6|          2.9|          3.6|          1.3|versicolor|
| 66|          6.7|          3.1|          4.4|          1.4|versicolor|
| 67|          5.6|          3.0|          4.5|          1.5|versicolor|
| 68|          5.8|          2.7|          4.1|          1.0|versicolor|
```

```
[ ] df_iris2.count()
```

37

```
[ ] spark.sql("""select Species , Count(*) from iristab group by Species""").show()
```

```
+-----+-----+
| Species|count(1)|
+-----+-----+
| virginica|      50|
| versicolor|    50|
| setosa|      50|
+-----+-----+
```

```
[ ] spark.sql(""" select Sepal_Length ,Sepal_Width , Species from iristab where Species='setosa' """).show()
```

```
+-----+-----+-----+
|Sepal_Length|Sepal_Width|Species|
+-----+-----+-----+
|          5.1|          3.5| setosa|
|          4.9|          3.0| setosa|
|          4.7|          3.2| setosa|
|          4.6|          3.1| setosa|
|          5.0|          3.6| setosa|
|          5.4|          3.9| setosa|
|          4.6|          3.4| setosa|
|          5.0|          3.4| setosa|
|          4.4|          2.9| setosa|
|          4.9|          3.1| setosa|
|          5.4|          3.7| setosa|
```

```
[ ] df_iris2 = spark.sql(""" select min(Sepal_Length) as min_sep_len_species , Species
                             from iristab group by Species""")
df_iris2.show()
```

```
+-----+-----+
|min_sep_len_species| Species|
+-----+-----+
|          4.9| virginica|
|          4.9|versicolor|
|          4.3| setosa|
+-----+-----+
```

```
[ ] df_iris.groupBy('Species').min('Sepal_Length').show()
```

```
+-----+-----+
| Species|min(Sepal_Length)|
+-----+-----+
| virginica|          4.9|
| versicolor|        4.9|
| setosa|          4.3|
+-----+-----+
```

```
[ ] spark.sql("""select Species , min(Petal_Length) as Min_Petal_Length ,
               max(Petal_Length) as Max_Petal_Length ,
               avg(Petal_Length) from iristab group by Species""").show()
```

```
+-----+-----+-----+-----+
| Species|Min_Petal_Length|Max_Petal_Length| avg(Petal_Length)|
+-----+-----+-----+-----+
| virginica|          4.5|          6.9|          5.552|
| versicolor|        3.0|          5.1|          4.26|
| setosa|          1.0|          1.9|1.4620000000000002|
+-----+-----+-----+-----+
```

```
[ ] spark.sql("""select mean(Petal_Length) as Avg_Petal_Length , Species from iristab group by
               Species having mean(Petal_Length)>4 """).show()
```

```
+-----+-----+
|Avg_Petal_Length| Species|
+-----+-----+
|          5.552| virginica|
|          4.26|versicolor|
+-----+-----+
```

```
[ ] df_iris.where('Species=="setosa").show()
```

```
↗ +---+-----+-----+-----+-----+-----+
  | ID|Sepal_Length|Sepal_Width|Petal_Length|Petal_Width|Species|
  +---+-----+-----+-----+-----+-----+
  | 1|      5.1|      3.5|      1.4|      0.2| setosa|
  | 2|      4.9|      3.0|      1.4|      0.2| setosa|
  | 3|      4.7|      3.2|      1.3|      0.2| setosa|
  | 4|      4.6|      3.1|      1.5|      0.2| setosa|
  | 5|      5.0|      3.6|      1.4|      0.2| setosa|
  | 6|      5.4|      3.9|      1.7|      0.4| setosa|
  | 7|      4.6|      3.4|      1.4|      0.3| setosa|
  | 8|      5.0|      3.4|      1.5|      0.2| setosa|
  | 9|      4.4|      2.9|      1.4|      0.2| setosa|
  |10|      4.9|      3.1|      1.5|      0.1| setosa|
  |11|      5.4|      3.7|      1.5|      0.2| setosa|
```

```
[ ] df_iris2 = df_iris.withColumnRenamed('Sepal_Length','New_Sepal_Length')
df_iris2.show()
```

```
↗ +---+-----+-----+-----+-----+-----+
  | ID|New_Sepal_Length|Sepal_Width|Petal_Length|Petal_Width|Species|
  +---+-----+-----+-----+-----+-----+
  | 1|      5.1|      3.5|      1.4|      0.2| setosa|
  | 2|      4.9|      3.0|      1.4|      0.2| setosa|
  | 3|      4.7|      3.2|      1.3|      0.2| setosa|
  | 4|      4.6|      3.1|      1.5|      0.2| setosa|
  | 5|      5.0|      3.6|      1.4|      0.2| setosa|
  | 6|      5.4|      3.9|      1.7|      0.4| setosa|
  | 7|      4.6|      3.4|      1.4|      0.3| setosa|
  | 8|      5.0|      3.4|      1.5|      0.2| setosa|
  | 9|      4.4|      2.9|      1.4|      0.2| setosa|
  |10|      4.9|      3.1|      1.5|      0.1| setosa|
  |11|      5.4|      3.7|      1.5|      0.2| setosa|
```

```
[ ] pd_iris = df_iris.toPandas()
pd_iris.head()
```

```
↗      ID  Sepal_Length  Sepal_Width  Petal_Length  Petal_Width  Species
0      1           5.1           3.5           1.4           0.2    setosa
1      2           4.9           3.0           1.4           0.2    setosa
2      3           4.7           3.2           1.3           0.2    setosa
3      4           4.6           3.1           1.5           0.2    setosa
4      5           5.0           3.6           1.4           0.2    setosa
```